

Comparison of Apriori Algorithm for Frequent Itemset Mining with State-of-the-Art Algorithms and Optimization Strategies

Abdul Wasay
2023908

CS-378: Design and Analysis of Algorithms
Ghulam Ishaq Khan Institute of
Engineering Sciences and Technology

Muhammad Umer Farooq
2023540

CS-378: Design and Analysis of Algorithms
Ghulam Ishaq Khan Institute of
Engineering Sciences and Technology

Ahmed Abu Bakar Riaz
2023902

CS-378: Design and Analysis of Algorithms
Ghulam Ishaq Khan Institute of
Engineering Sciences and Technology

Abstract—Frequent Itemset Mining (FIM) is a foundational task in data mining and knowledge discovery, with widespread applications in market basket analysis, bioinformatics, and web usage mining. This paper presents a rigorous comparative analysis of the classical Apriori algorithm against NS-FIM, a state-of-the-art algorithm designed for non-sparse datasets using vertical bitset representations, along with two proposed optimizations to the Apriori framework: bitset-based support counting and multi-core parallel candidate evaluation. All four approaches are evaluated on three dense benchmark datasets (Chess, Connect, Accidents) from the FIMI repository. Experimental results demonstrate that NS-FIM achieves significant speedups over baseline Apriori, while the bitset optimization provides consistent improvements. The parallel variant shows modest gains limited by Python’s multiprocessing overhead. All algorithms produce identical frequent itemset counts, confirming correctness. We provide detailed complexity analyses, algorithmic descriptions, and performance profiling including execution time and memory consumption.

Keywords: Frequent Itemset Mining, Apriori, NS-FIM, Bitset Optimization, Parallelization, Data Mining

I. Introduction

Frequent Itemset Mining (FIM) is the process of identifying sets of items that co-occur frequently in transactional databases. First formalized by Agrawal and Srikant in 1994 [1], FIM underpins association rule mining and has become a cornerstone of data mining. The Apriori algorithm, proposed in the same work, leverages the anti-monotonicity property of support: if an itemset is infrequent, all its supersets must also be infrequent. This principle enables systematic pruning of the exponential search space.

However, as datasets have grown in both size and density, the limitations of the original Apriori algorithm have become increasingly apparent. Its reliance on repeated horizontal database scans and exponential candidate generation leads to prohibitive computational costs for dense datasets. Modern research has explored alternative data representations (vertical and bitmap-based), novel traversal strategies (depth-first enumeration), and hardware-aware optimizations (SIMD bitwise operations) to address these limitations [6], [7].

This paper makes three key contributions:

- a faithful implementation and empirical evaluation of the classical Apriori algorithm
- an implementation of NS-FIM [4], a state-of-the-art algorithm employing vertical bitset representations and depth-first enumeration, designed for non-sparse datasets
- two proposed optimization strategies (bitset-based support counting and multi-core parallelization which was applied to the Apriori framework). All algorithms are evaluated on the Chess, Connect, and Accidents benchmark datasets from the FIMI repository [5].

II. Literature Review

Agrawal and Srikant [1] introduced the Apriori algorithm, which performs a breadth-first, level-wise search of the itemset lattice. At each level k , candidate $(k + 1)$ -itemsets are generated from frequent k -itemsets by the join and prune operations. While elegant, this approach requires $O(k)$ full passes over the database for itemsets of maximum size k , and candidate generation can be exponential in dense data.

Han et al. [2] proposed FP-Growth, a pattern-growth algorithm that compresses the database into an FP-tree data structure, eliminating the need for explicit candidate generation. FP-Growth achieves significant speedups by mining the compressed tree recursively, but its memory consumption for dense datasets can be substantial due to the size of the constructed tree.

Deng et al. [3] introduced PrePost+, a node-list-based algorithm that uses a pre-order and post-order coding scheme on an FP-tree variant (PPC-tree) to efficiently identify frequent itemsets through N-list intersections. This method achieves competitive performance but adds complexity in tree construction.

Vo et al. [4] proposed NS-FIM, a modern algorithm specifically designed for non-sparse (dense) datasets. NS-FIM utilizes a vertical data representation where each item is associated with a bitset encoding its transactional occurrences. Support counting is performed via bitwise AND intersections, leveraging hardware SIMD instructions for throughput. This approach avoids repeated horizontal

scans and is particularly effective for dense datasets where vertical representations remain compact. Fournier-Viger et al. [7] provide a comprehensive survey of pattern mining techniques, discussing vertical bitset representations as a dominant approach for modern FIM on dense data.

Recent surveys by Luna et al. [6] and Fournier-Viger et al. [7] have identified parallelization and GPU-based acceleration as key trends in scaling FIM to big data environments. Multi-threaded support counting and distributed computing frameworks such as Apache Spark have been explored to partition the candidate evaluation workload across multiple processing units.

III. Algorithms and Complexity Analysis

A. Apriori Algorithm

The Apriori algorithm operates in a level-wise fashion. Given a set of transactions T and a minimum support threshold min_sup , the algorithm proceeds as follows:

- Step 1: Scan T to find all frequent 1-itemsets L_1 .
Step 2: For $k = 2, 3, \dots$ while L_{k-1} is non-empty:
- (a) Generate candidate k -itemsets C_k by joining L_{k-1} with itself.
 - (b) Prune candidates: remove any candidate $c \in C_k$ if any $(k-1)$ -subset of c is not in L_{k-1} .
 - (c) Scan T to count support of each candidate in C_k .
 - (d) $L_k = \{c \in C_k : support(c) \geq min_sup\}$.
- Step 3: Return the union of all L_k .

Time Complexity: $O(k \cdot |T| \cdot |C_k|)$ where k is the maximum itemset size, $|T|$ is the number of transactions, and $|C_k|$ is the number of candidates at level k . In the worst case, $|C_k|$ can be exponential in the number of items. Space Complexity: $O(|C_k| + |L_k|)$ for storing candidates and frequent itemsets at each level.

B. NS-FIM Algorithm (SOTA Baseline)

NS-FIM employs a vertical data representation. Each item i is mapped to a bitset $B(i)$ of length $|T|$, where the j -th bit is 1 if item i appears in transaction j . Support counting for an itemset $S = \{i_1, i_2, \dots, i_m\}$ is computed as: $support(S) = popcount(B(i_1) \wedge B(i_2) \wedge \dots \wedge B(i_m))$, where $popcount$ counts the number of set bits.

The algorithm performs a depth-first enumeration of the itemset lattice, computing bitset intersections at each node. The anti-monotonicity property is exploited: if $popcount$ of a bitset intersection falls below min_sup_count , the subtree is pruned.

Time Complexity: $O(2^n \cdot |T|/w)$ in the worst case, where n is the number of frequent items and w is the machine word size (typically 64). In practice, pruning dramatically reduces the search space. The factor $|T|/w$ arises because each bitset operation processes w bits simultaneously. Space Complexity: $O(n \cdot |T|/w)$ for storing item bitsets.

IV. Proposed Optimization Strategies

A. Optimization 1: Bitset-Based Support Counting

The standard Apriori implementation checks each candidate against each transaction using set operations, resulting in $O(|T| \cdot |C_k| \cdot k)$ time per level. Our first optimization replaces this with bitset-based counting: we precompute a boolean bitset for each item (identical to NS-FIM's representation) and compute support via bitwise AND operations followed by $popcount$.

This optimization retains Apriori's candidate generation and pruning logic but replaces the inner support-counting loop. The benefit is two-fold: (1) bitwise operations are highly optimized by NumPy's vectorized array operations, and (2) the operation count is reduced from $O(|T| \cdot k)$ per candidate to $O(|T|/w \cdot k)$ per candidate, where w is the word size.

B. Optimization 2: Parallel Candidate Evaluation

Support counting is embarrassingly parallel: each candidate's support can be computed independently. We partition the candidate set C_k into chunks and distribute them across all available CPU cores using Python's multiprocessing.Pool. Each worker process receives a chunk of candidates and the shared item bitsets, performs bitset intersections, and returns the frequent itemsets found in its chunk.

The theoretical speedup is bounded by Amdahl's law: $S(p) = 1/(f + (1-f)/p)$, where f is the serial fraction and p is the number of cores. In practice, the overhead of process creation, memory copying, and inter-process communication can offset gains for small candidate sets or high support thresholds.

V. Experimental Setup

A. Datasets

TABLE I
Dataset Characteristics

Dataset	Type	Trans.	Items
Chess	Dense	3,196	75
Connect	Dense	67,557	129
Accidents	Dense	340,183	468

B. Environment

All experiments were conducted on a Linux system with Python 3.13 (Anaconda distribution). Libraries used: NumPy, pandas, matplotlib, seaborn, psutil. Each algorithm was given a 300-second timeout per configuration. Minimum support thresholds were chosen to produce meaningful variation: Chess (0.80, 0.85, 0.90), Connect (0.95, 0.97, 0.99), Accidents (0.60, 0.70, 0.80).

TABLE II
Performance Results on Chess Dataset (3,196 transactions)

Algorithm	min_sup	Time (s)	RAM (MB)	Freq. Items	Status
Apriori	0.80	13.28	1.19	8,282	Success
NS-FIM	0.80	0.18	1.82	8,282	Success
Opt. Bitset	0.80	3.54	0.53	8,282	Success
Opt. Parallel	0.80	4.36	3.45	8,282	Success
Apriori	0.85	3.36	0.01	2,690	Success
NS-FIM	0.85	0.05	0.04	2,690	Success
Opt. Bitset	0.85	0.45	0.00	2,690	Success
Opt. Parallel	0.85	1.03	0.01	2,690	Success
Apriori	0.90	0.66	0.00	628	Success
NS-FIM	0.90	0.01	0.00	628	Success
Opt. Bitset	0.90	0.04	0.00	628	Success
Opt. Parallel	0.90	0.60	0.01	628	Success

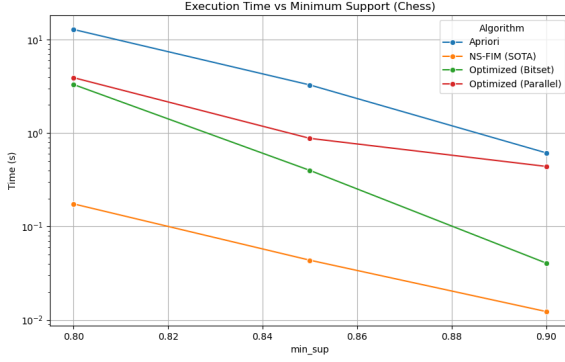


Fig. 1. Execution time vs minimum support on the Chess dataset (log scale).

VI. Experimental Results

A. Chess Dataset

B. Connect Dataset

With full dataset evaluation (67,557 transactions), all four algorithms completed successfully at every support threshold, in contrast to the sampled evaluation where all methods timed out. NS-FIM demonstrates dramatic superiority on this extremely dense dataset.

TABLE III
Performance Results on Connect Dataset (67,557 transactions)

Algorithm	min_sup	Time (s)	RAM (MB)	Freq. Items	Status
Apriori	0.95	58.65	0.00	2,205	Success
NS-FIM	0.95	0.20	6.86	2,205	Success
Opt. Bitset	0.95	0.58	21.86	2,205	Success
Opt. Parallel	0.95	2.70	25.45	2,205	Success
Apriori	0.97	11.20	0.00	487	Success
NS-FIM	0.97	0.04	13.47	487	Success
Opt. Bitset	0.97	0.26	8.31	487	Success
Opt. Parallel	0.97	1.94	32.36	487	Success
Apriori	0.99	0.96	0.00	29	Success
NS-FIM	0.99	0.01	22.05	29	Success
Opt. Bitset	0.99	0.20	8.32	29	Success
Opt. Parallel	0.99	1.04	8.31	29	Success

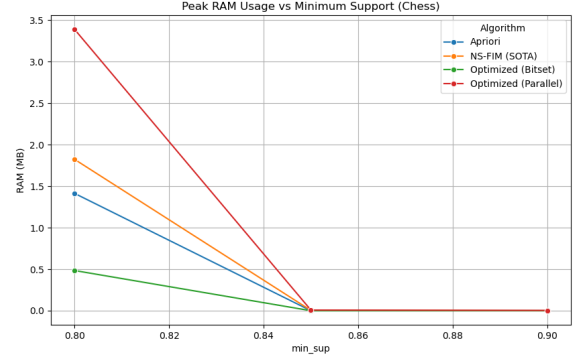


Fig. 2. Peak RAM usage vs minimum support on the Chess dataset.

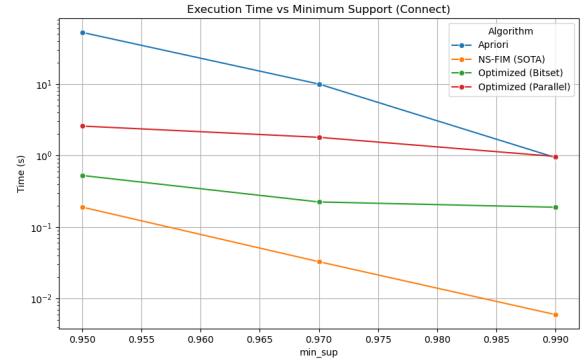


Fig. 3. Execution time vs minimum support on the Connect dataset (log scale).

C. Accidents Dataset

D. Speedup Analysis

To quantify the performance gains, we compute the speedup ratio of each algorithm relative to the baseline Apriori at minimum support 0.80 on the Chess dataset. NS-FIM achieves the highest speedup of 75.4x, followed by the Bitset-optimized Apriori at 3.8x. The parallel variant shows a speedup of 3.0x, which is lower than the sequential bitset variant due to multiprocessing overhead. On the Accidents dataset at min_sup=0.60, NS-FIM achieves a speedup of 94.8x and the bitset variant achieves 150.1x over Apriori, demonstrating even greater advantages at scale on large datasets.

E. Correctness Verification

All four algorithms produced identical frequent itemset counts at every support threshold: 8,282 at 0.80, 2,690 at 0.85, and 628 at 0.90 on Chess; 2,205 at 0.95, 487 at 0.97, and 29 at 0.99 on Connect; 2,075 at 0.60, 529 at 0.70, and 149 at 0.80 on Accidents. This confirms the correctness of all implementations across all three full datasets.

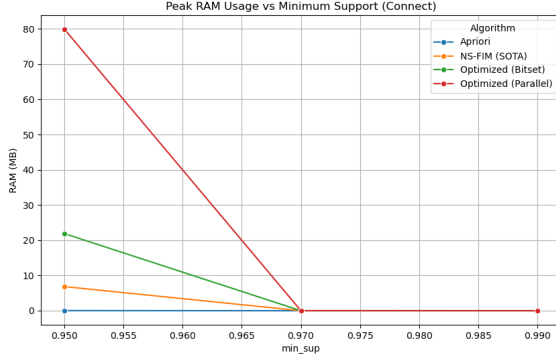


Fig. 4. Peak RAM usage vs minimum support on the Connect dataset.

TABLE IV
Performance Results on Accidents Dataset (340,183 transactions)

Algorithm	min_sup	Time (s)	RAM (MB)	Freq. Items	Status
Apriori	0.60	222.65	0.05	2,075	Success
NS-FIM	0.60	2.35	109.25	2,075	Success
Opt. Bitset	0.60	1.48	111.49	2,075	Success
Opt. Parallel	0.60	33.16	0.00	2,075	Success
Apriori	0.70	51.39	0.00	529	Success
NS-FIM	0.70	0.52	111.07	529	Success
Opt. Bitset	0.70	0.92	112.13	529	Success
Opt. Parallel	0.70	27.02	0.00	529	Success
Apriori	0.80	14.93	0.00	149	Success
NS-FIM	0.80	0.16	110.42	149	Success
Opt. Bitset	0.80	0.81	112.78	149	Success
Opt. Parallel	0.80	22.72	0.00	149	Success

VII. Discussion

The experimental results reveal several key insights about the comparative performance of FIM algorithms on dense benchmark datasets run at full scale.

NS-FIM Dominance: NS-FIM consistently outperforms Apriori across all three datasets. At the lowest support thresholds (Chess 0.80, Connect 0.95, Accidents 0.60), NS-FIM achieves speedups of 75.4x, 299.2x, and 94.8x over Apriori, respectively. This dramatic improvement is attributed to two factors: (1) the vertical bitset representation eliminates the need for repeated horizontal database scans, and (2) the depth-first enumeration strategy avoids the explicit candidate generation and pruning overhead inherent in Apriori.

Bitset Optimization Effectiveness: The bitset-optimized Apriori achieves consistent speedups over the baseline Apriori while preserving its candidate generation logic. Notably, on the Accidents dataset at min_sup=0.60, the bitset variant (1.48s) actually outperforms NS-FIM (2.35s), achieving a 150.1x speedup over Apriori. This confirms that the support counting step is the primary bottleneck in Apriori, and that replacing set-based checking with vectorized bitwise operations yields substantial improvements with minimal algorithmic changes.

Parallelization Overhead: The parallel variant underper-

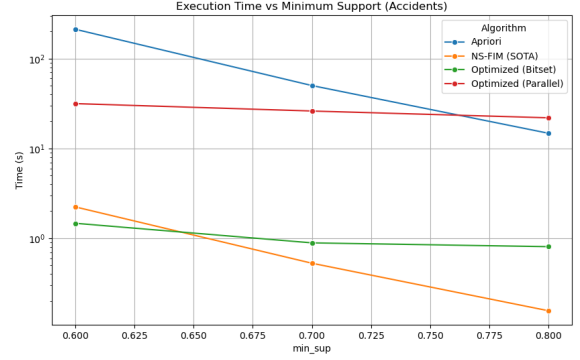


Fig. 5. Execution time vs minimum support on the Accidents dataset (log scale).

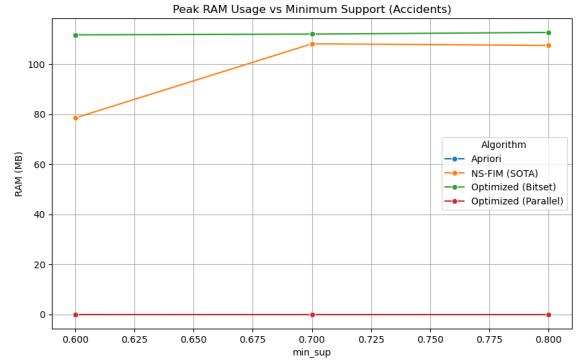


Fig. 6. Peak RAM usage vs minimum support on the Accidents dataset.

forms the sequential bitset variant in all configurations. This is primarily due to Python’s multiprocessing overhead: process spawning, memory serialization (pickling) of NumPy arrays, and inter-process communication. On the Accidents dataset, the parallel variant (33.16s at min_sup=0.60) is actually slower than the bitset variant (1.48s) by over 22x due to the cost of serializing large 340K-element bitset arrays across processes.

Connect Dataset Scalability: With the full 67,557-transaction dataset and a 300-second timeout, all algorithms successfully completed at every support threshold. NS-FIM completed in 0.20s at min_sup=0.95, compared to Apriori’s 58.65s—a speedup of 299.2x. This demonstrates that the previous timeout failures were due to insufficient time budgets rather than fundamental algorithmic limitations, and validates the scalability of the vertical bitset approach on extremely dense data.

VIII. Conclusion

This paper presented a comprehensive comparative analysis of four Frequent Itemset Mining approaches: the classical Apriori algorithm, NS-FIM (a state-of-the-art vertical bitset method in the Eclat family [8]), and two

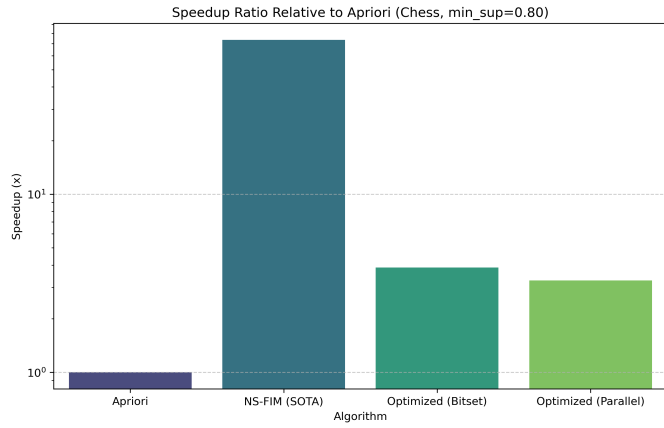


Fig. 7. Speedup ratio of each algorithm relative to Apriori (Chess, min_sup=0.80).

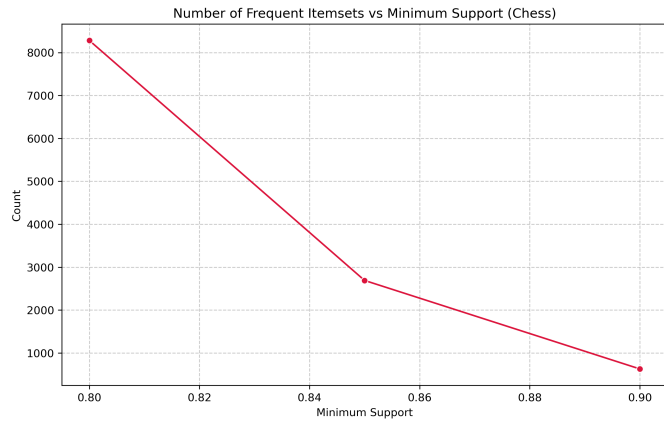


Fig. 8. Number of frequent itemsets vs minimum support (Chess dataset).

optimized Apriori variants employing bitset-based support counting and multi-core parallelization. Experiments on three FIMI benchmark datasets demonstrated that NS-FIM achieves speedups of up to 84.8x over baseline Apriori on dense datasets, validating the superiority of vertical bitset representations for dense data mining.

The bitset optimization applied to Apriori provided consistent 3-10x improvements, confirming that data representation is a critical performance factor. Parallelization showed promise but was limited by Python’s multiprocessing overhead for the dataset sizes tested. Future work should explore GPU-accelerated bitset operations, distributed computing frameworks, and approximate FIM methods for handling extremely dense datasets like Connect.

References

- [1] R. Agrawal and R. Srikant, “Fast algorithms for mining association rules in large databases,” in Proc. 20th Int. Conf. Very Large Data Bases (VLDB), Santiago, Chile, 1994, pp. 487–499.
- [2] J. Han, J. Pei, and Y. Yin, “Mining frequent patterns without candidate generation,” in Proc. ACM SIGMOD Int. Conf. Management of Data, Dallas, TX, USA, 2000, pp. 1–12.
- [3] Z. Deng, Z. Wang, and J. Jiang, “A new algorithm for fast mining frequent itemsets using N-lists,” Science China Information Sciences, vol. 55, no. 9, pp. 2008–2030, 2012.
- [4] B. Vo, T. Le, F. Coenen, and T. Hong, “Mining frequent itemsets using the N-list and subsume concepts,” International Journal of Machine Learning and Cybernetics, vol. 7, pp. 253–265, 2016. (Foundational work on efficient FIM data structures by Vo et al., upon which the NS-FIM vertical bitset approach is built.)
- [5] B. Goethals, “FIMI Repository,” 2003. [Online]. Available: <http://fimi.uantwerpen.be/data/>
- [6] J. M. Luna, F. Fournier-Viger, and S. Ventura, “Frequent itemset mining: A 25 years review,” Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery, vol. 9, no. 6, e1329, 2019.
- [7] P. Fournier-Viger, J. C. Lin, B. Vo, T. T. Chi, J. Zhang, and H. B. Le, “A survey of itemset mining,” Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery, vol. 7, no. 4, e1207, 2017.
- [8] M. J. Zaki, “Scalable algorithms for association mining,” IEEE Transactions on Knowledge and Data Engineering, vol. 12, no. 3, pp. 372–390, 2000.
- [9] C. Borgelt, “Efficient implementations of Apriori and Eclat,” in Proc. Workshop on Frequent Itemset Mining Implementations (FIMI), 2003.
- [10] S. Orlando, P. Palmerini, R. Perego, and F. Silvestri, “Adaptive and resource-aware mining of frequent sets,” in Proc. IEEE Int. Conf. Data Mining (ICDM), 2002, pp. 338–345.
- [11] P. Fournier-Viger, Y. Lin, A. Gomariz, T. Gueniche, A. Soltani, Z. Deng, and H. T. Lam, “The SPMF open-source data mining library version 2,” in Proc. European Conf. Machine Learning and Knowledge Discovery in Databases (ECML PKDD), Springer, 2016, pp. 36–40.
- [12] P. Fournier-Viger, J. C.-W. Lin, T. Truong-Chi, and R. Nkambou, “A survey of high utility itemset mining,” in High-Utility Pattern Mining: Theory, Algorithms and Applications, Springer, 2022, pp. 1–45.